Topic 9B: Channel Coding

Reliable Transmission over Noisy Channels
CSE 4405: Data and Telecommunications

Md. Tanvir Hossain Saikat, Jr. Lecturer

Islamic University of Technology (IUT)

From Compression to Protection

Channel-coding question

After source coding produces a bitstream, how can those bits be transmitted reliably through a channel that may corrupt them?

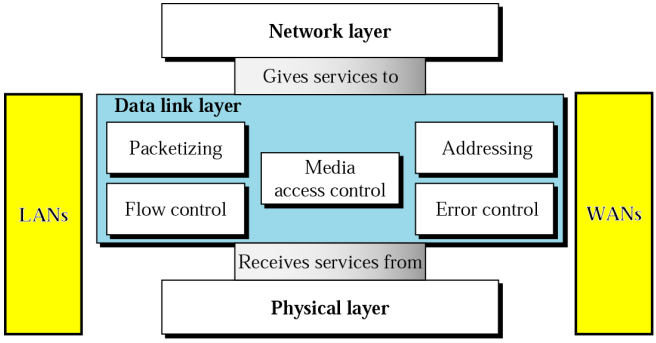
- Real channels suffer from noise, interference, fading, attenuation, and distortion.
- Source coding removes avoidable redundancy; channel coding deliberately adds structured redundancy.
- Error-control coding is already part of the CSE 4405 syllabus through block codes, Hamming codes, cyclic codes, and convolutional codes.

Learning Objectives

1. Distinguish error detection, error correction, FEC, and retransmission.
2. Use parity and repetition codes to explain the basic redundancy idea.
3. Compute code rate $R = k/n$ and Hamming distance for small binary codes.
4. Use $d_{\min}$ to reason about detection and correction capability.
5. Explain the roles of Hamming codes, cyclic codes, and CRC.
6. Encode a short message with a rate-$1/2$ convolutional encoder, place it in the general (n, k, m) family, and read its state diagram and trellis.
7. Run the Viterbi algorithm by hand on a corrupted received word and recover the message.
8. Explain how free distance, puncturing, and soft-decision decoding adapt one convolutional engine to different links and rates.

Where Error Control Fits in the Course

- The data-link layer uses error control for local frame reliability.
- Wireless/mobile systems often need forward correction because retransmission may cost delay and capacity.
- This module links Topic 9 error-control methods to the larger information pipeline.



Error control sits in the data-link layer of the protocol stack.

Detection vs. Correction

Error detection

The receiver decides that at least one error has occurred. It does not need to know exactly which bit is wrong.

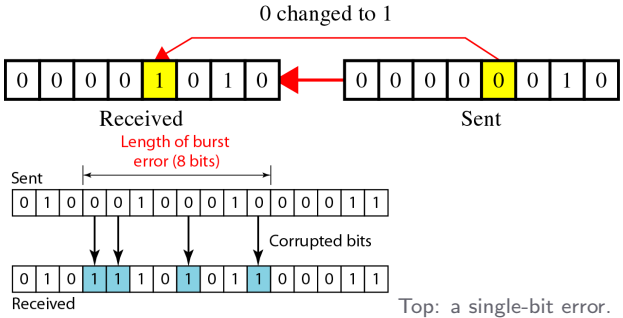
Error correction

The receiver must infer the original codeword, which usually means identifying the likely error pattern.

Technique	Purpose	Examples
Detection	Reject or request repair	Parity, checksum, CRC
Correction	Repair without immediate retransmission	Hamming, convolutional

Single-Bit and Burst Errors

- A single-bit error changes one bit in the data unit.
- A burst error affects a span from the first corrupted bit to the last corrupted bit; bits inside the span do not all need to change.
- A code that handles isolated errors may be weak against bursty channels.

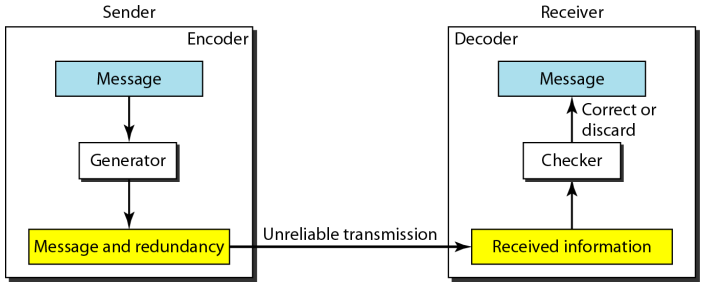


Top: a single-bit error.

Bottom: a burst error spanning several bits.

General Encoder-Decoder View

- The sender maps a dataword into a longer codeword.
- The channel may corrupt the codeword.
- The receiver checks whether the received word is valid or close to a valid codeword.



Encoder, noisy channel, and decoder as a single block diagram.

Code Rate: The Cost of Protection

Code rate

If k data bits are encoded into n transmitted bits, the code rate is

$$R = \frac{k}{n}.$$

- Lower rate means more redundancy per data bit.
- More redundancy may improve reliability, but it consumes bandwidth or time.
- The Hamming $(7, 4)$ code has $R = 4/7$.

Simple Parity Check

Data bits: 1011. The number of ones is 3, which is odd.

$$\text{Even parity bit} = 1, \qquad \text{transmitted word} = 10111.$$

- A single-bit error changes the parity from even to odd, so it is detected.
- Parity does not identify the location of the wrong bit.
- Any even number of bit flips can pass the parity check.

Parity Check as a Code

- A parity bit makes all valid codewords have even weight for even parity.
- The receiver tests whether the received word satisfies the parity rule.
- Detection is based on membership in the valid-codeword set.

Datawords	*Codewords*	*Datawords*	*Codewords*
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Datawords and codewords of the simple parity-check code.

Repetition Code

Threefold repetition

Encode $0 \to 000$, $1 \to 111$. Decode by majority vote.

Example

received 101

two ones, one zero $\Rightarrow \widehat{x} = 1$.

Cost

$$R = \frac{1}{3}.$$

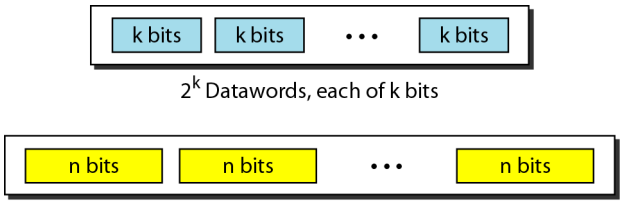
Only one information bit is sent for every three transmitted bits.

Why it is useful pedagogically

Repetition shows correction by distance without hiding the idea inside algebra, but it is inefficient.

Block-Code Terminology

- A dataword has k bits.
- A codeword has n bits.
- The encoder maps 2^k possible datawords into 2^k valid codewords.
- The full n-bit space has 2^n possible received words.



Block-code expansion from datawords to codewords.

Hamming Distance

Definition

The Hamming distance $d(x, y)$ between two equal-length binary strings is the number of positions in which they differ.

Example

$$x = 101100, \qquad y = 100110.$$

$$\begin{array}{cccccc} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \end{array} \quad \Rightarrow \quad d(x, y) = 2.$$

Geometric reading

Distance tells us how many bit flips are needed to move from one word to another.

Minimum Hamming Distance

Definition

The minimum distance $d_{\min}$ of a code is the smallest Hamming distance between any two distinct valid codewords.

- If valid codewords are far apart, corrupted words are less likely to be confused with another valid codeword.
- $d_{\min}$ is the key design number for detection and correction capability.
- The receiver can often reason by nearest valid codeword when the channel creates few bit errors.

Detection and Correction Rules

Detect up to s errors if $d_{\min} \geq s + 1$.

Correct up to t errors if $d_{\min} \geq 2t + 1$.

Equivalent forms

$$s_{\max} = d_{\min} - 1, \qquad t_{\max} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor.$$

For $d_{\min} = 3$

Detect up to 2 bit errors, or correct up to 1 bit error.

Hamming $(7,4)$ Code

Core fact

The classic Hamming $(7,4)$ code maps 4 data bits into 7 transmitted bits by adding 3 parity bits. It has $d_{\min} = 3$.

$$k = 4, \qquad n = 7, \qquad R = \frac{4}{7}.$$

$$t_{\max} = \left\lfloor \frac{3-1}{2} \right\rfloor = 1.$$

Meaning

One bit may be corrupted anywhere in the 7-bit codeword and the receiver can still identify the bit position to flip.

Hamming $(7, 4)$: Codeword Table

- Four data bits produce sixteen possible datawords.
- The code maps each dataword to one 7-bit codeword.
- The table lets students see that a code is a set, not just a formula.

Datawords	*Codewords*	*Datawords*	*Codewords*
0000	0000000	1000	1000110
0001	0001101	1001	1001011
0010	0010111	1010	1010001
0011	0011010	1011	1011100
0100	0100011	1100	1100101
0101	0101110	1101	1101000
0110	0110100	1110	1110010
0111	0111001	1111	1111111

All sixteen datawords and their $C(7, 4)$ codewords.

Hamming Decoder: Syndrome Idea

- A syndrome is a compact pattern computed from parity checks.
- Zero syndrome means the received word satisfies all checks.
- A nonzero syndrome points to a likely error position for single-error correction.

Syndrome	000	001	010	011	100	101	110	111
Error	None	q_0	q_1	b_2	q_2	b_0	b_3	b_1

Each syndrome value maps to a decoder decision.

Cyclic Codes and CRC

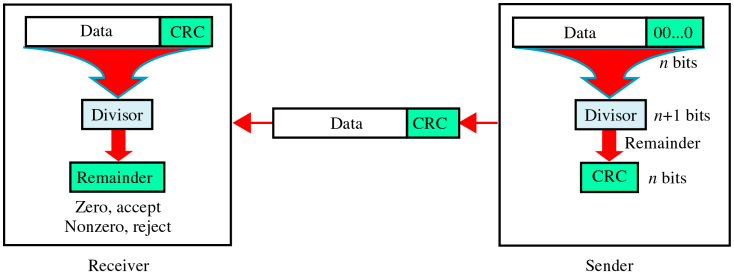
CRC idea

A cyclic redundancy check treats a bit sequence as a polynomial over binary coefficients, divides by a generator polynomial, and appends the remainder as check bits.

- The receiver repeats the division and checks the remainder.
- CRC is primarily an error-detection method.
- The full polynomial algebra can be learned after students understand the sender/receiver consistency check.

CRC Encoder-Decoder View

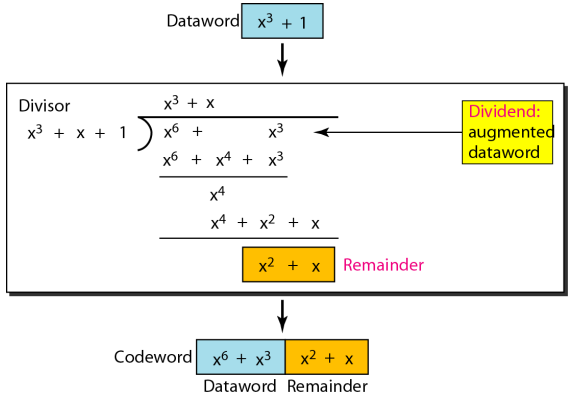
- The sender computes a remainder from the dataword and generator.
- The codeword is data plus remainder.
- The receiver divides the received word by the same generator.



CRC generation at the sender and checking at the receiver.

CRC Polynomial Division

- Binary subtraction is XOR.
- The remainder length is one less than the generator length.
- Appending the remainder makes the transmitted polynomial divisible by the generator.



Worked binary polynomial division producing the CRC remainder.

CRC vs. Hamming: Different Jobs

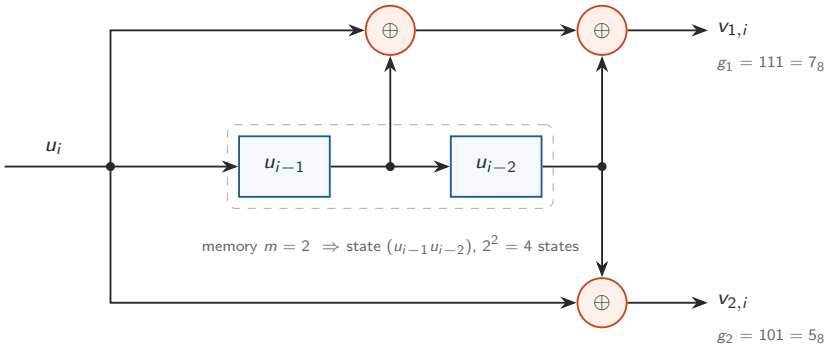
Method	Main strength	Typical use in this course
Parity	Very simple detection	Introductory consistency check
Hamming code	Single-bit correction with modest redundancy	Forward correction example
CRC	Strong burst-error detection for frames/storage	Data-link error detection

Design reading

A detector can be very strong without being a corrector. Correction requires enough structure to identify an error pattern, not only reject invalid words.

Convolutional Codes

- Unlike block codes, a convolutional encoder processes a stream.
- Output bits depend on the current input and previous input bits stored in memory.



Rate-1/2 encoder $(7, 5)_8$. A dot is a tap: every wire leaving it carries that bit. The top adder sums all three taps; the bottom adder skips the middle one. Every frame that follows uses exactly this encoder.

Block Code vs. Convolutional Code

	Block code (Hamming, CRC)	**Convolutional code**
Input handling	Fixed k-bit blocks	k bits per step, without end
Memory	Each block encoded independently	Output depends on the last m bits too
Encoder state	Nothing carried between blocks	2^m states carried across time
Decoding	Syndrome or table lookup	Best path through a trellis
Natural fit	Frames, files, storage	Continuous radio and satellite streams

One-line difference

A block encoder has amnesia between blocks; a convolutional encoder remembers, so every transmitted bit carries evidence about several information bits.

Anatomy of the Encoder

- **Memory** $m = 2$: two past bits are stored.
- **Constraint length** $K = m + 1 = 3$: each output bit depends on a window of 3 input bits.
- **State** $= (u_{i-1}u_{i-2})$, so the encoder has $2^m = 4$ states.
- **Rate** $R = k/n = 1/2$: $k = 1$ bit enters and $n = 2$ bits leave at every step.

$$v_{1,i} = u_i \oplus u_{i-1} \oplus u_{i-2}$$

$$v_{2,i} = u_i \oplus u_{i-2}$$

Generator vectors

$g_1 = (1, 1, 1) = 7_8$ and $g_2 = (1, 0, 1) = 5_8$, so this is the $(7, 5)_8$, $K = 3$ code. A 1 in position l of g_j means "output j taps the bit that entered l steps ago".

Where the Name "Convolutional" Comes From

Feed a single 1 followed by zeros into the empty encoder and watch each output line.

Input u_i	1	0	0	0	0
Output v_1	1	1	1	0	0
Output v_2	1	0	1	0	0

- The v_1 response is $111 = g_1$ and the v_2 response is $101 = g_2$: the impulse response spells out the generators.
- Each output is the modulo-2 convolution of the input with a generator, hence the name.

$$v_{j,i} = \bigoplus_{l=0}^{m} g_j[l]\, u_{i-l} \qquad (j = 1, \ldots, n)$$

The General (n, k, m) Encoder

- Our encoder is one member of a family: in general k bits enter and n leave at every step.
- Input stream p has its own register of length m_p; output j sums a chosen subset of the stored bits, selected by the kn generators $g_j^{(p)}$.
- **Memory order** $m = \max_p m_p$, **total memory** $\nu = \sum_p m_p$: the state is all ν stored bits.

$$v_{j,i} = \bigoplus_{p=1}^{k} \bigoplus_{l=0}^{m_p} g_j^{(p)}[l]\; u_{i-l}^{(p)}$$

$$R = \frac{k}{n}, \quad K = m + 1, \quad 2^{\nu} \text{ states}$$

	k	n	m	ν	R	K	**States**
General	k	n	$\max_p m_p$	$\sum_p m_p$	k/n	$m+1$	2^{ν}
Ours $(7,5)_8$	1	2	2	2	1/2	3	4

Why $k = 1$ in practice

A $k > 1$ encoder puts 2^k branches out of every state. Real systems keep $k = 1$ and reach 2/3 or 3/4 by *puncturing*, leaving one trellis to build.

Worked Example: Encoding $\mathbf{u} = 101$

Start with empty memory (state 00) and append $m = 2$ flush bits so the encoder finishes back in state 00. Note $\mathbf{u}$ is the message and m is the memory: they are different objects.

Step i	**Input** u_i	**State** $(u_{i-1}u_{i-2})$	$v_1 = u_i \oplus u_{i-1} \oplus u_{i-2}$	$v_2 = u_i \oplus u_{i-2}$	**Next state**
1	1	00	$1 \oplus 0 \oplus 0 = 1$	$1 \oplus 0 = 1$	10
2	0	10	$0 \oplus 1 \oplus 0 = 1$	$0 \oplus 0 = 0$	01
3	1	01	$1 \oplus 0 \oplus 1 = 0$	$1 \oplus 1 = 0$	10
4	0 (flush)	10	$0 \oplus 1 \oplus 0 = 1$	$0 \oplus 0 = 0$	01
5	0 (flush)	01	$0 \oplus 0 \oplus 1 = 1$	$0 \oplus 1 = 1$	00

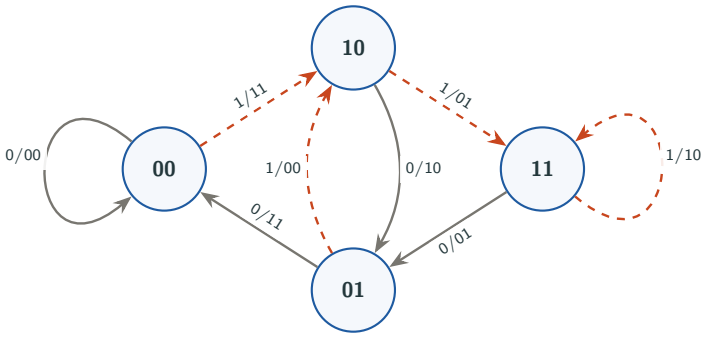
Transmitted codeword

$\mathbf{v} = 11\ 10\ 00\ 10\ 11$

Three message bits became ten transmitted bits, and the encoder ended in state 00 as required.

The Encoder as a State Machine

- A state is the pair of remembered bits $(u_{i-1}u_{i-2})$.
- **Next state** $= (u_i\, u_{i-1})$: the input shifts in, u_{i-2} drops out.
- Each arrow is labelled $u_i / v_1 v_2$.
- **Grey solid** = input 0; **red dashed** = input 1.
- Two arrows leave and two enter every state. That is what makes decoding tractable.



Branch Table: Every Transition in One Place

Both tables hold the same eight branches — $v_1 = u_i \oplus u_{i-1} \oplus u_{i-2}$ and $v_2 = u_i \oplus u_{i-2}$, evaluated for all four states and both inputs. The right one is the left one read backwards: the two branches add–compare–select must weigh, and the u_i traceback reads off the winner. Every branch metric in the Viterbi example is a lookup here.

Forward: what the encoder does

State	$u_i = 0$ emits→next	$u_i = 1$ emits→next
00	$00 \to 00$	$11 \to 10$
01	$11 \to 00$	$00 \to 10$
10	$10 \to 01$	$01 \to 11$
11	$01 \to 01$	$10 \to 11$

Reverse: what the decoder compares

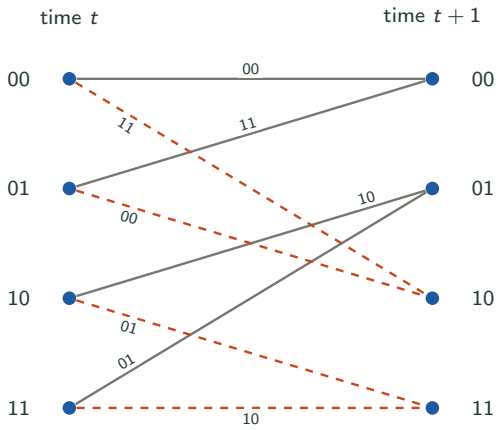
Into	Predecessor A from (emits, u_i)	Predecessor B from (emits, u_i)
00	00 (00, 0)	01 (11, 0)
01	10 (10, 0)	11 (01, 0)
10	00 (11, 1)	01 (00, 1)
11	10 (01, 1)	11 (10, 1)

A pattern worth noticing

Because the next state is $(u_i\, u_{i-1})$, the input is the leading bit of the destination: every $u_i = 0$ branch lands in 00 or 01, every $u_i = 1$ branch in 10 or 11.

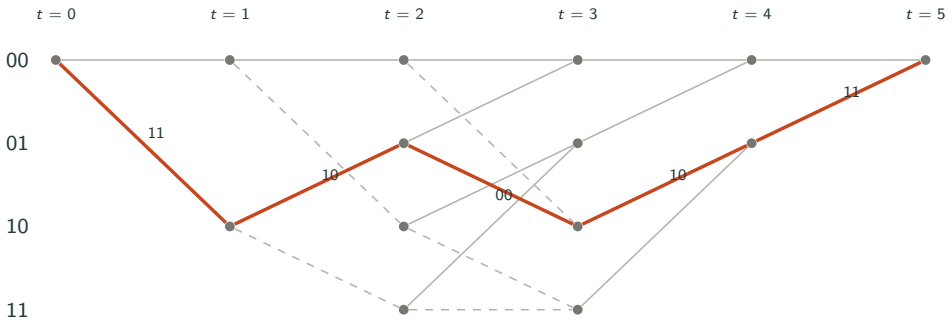
From State Diagram to Trellis

- A trellis is the state diagram unrolled in time: one column of states per time step.
- Every branch carries the two bits the encoder would transmit.
- The encoder walks one path through this trellis; the decoder has to guess which one.



One trellis stage. **Grey solid** = input 0, **red dashed** = input 1; each label is the pair that would be transmitted.

Trellis for $\mathbf{u} = 101$: The Encoder's Path



Terminated trellis for a 3-bit message. The highlighted path is $00 \to 10 \to 01 \to 10 \to 01 \to 00$, which emits 11 10 00 10 11. The last two stages have only input-0 branches because the flush bits are known.

Termination: Why the Two Flush Bits?

- Flushing forces the path to end at state 00, so the decoder knows where the trellis must close.
- Without it, the final message bits are protected by fewer output bits than the earlier ones.
- The cost is m extra stages, which matters only for short messages.

$$R_{\text{eff}} = \frac{L}{2\,(L+m)}$$

L info bits	Coded bits	R_{eff}
3	10	0.300
20	44	0.455
100	204	0.490
1000	2004	0.499

Alternative

Tail-biting starts the memory with the last m message bits, so the path begins and ends in the same state. No flush overhead; LTE uses it on control channels.

Reading the Termination Cost

Count clock cycles: pushing L message bits through *and* emptying the register takes $L + m$ clocks, each emitting n bits. Only L carry information, so the denominator is $n(L + m)$, not $nL + m$ — a flush stage emits a full pair like any other.

$$R_{\text{eff}} = \frac{L}{n\,(L+m)} = R \cdot \underbrace{\frac{L}{L+m}}_{\text{termination penalty}}, \qquad \frac{R - R_{\text{eff}}}{R} = \frac{m}{L+m}$$

- The nominal rate is untouched — it is *multiplied* by the fraction of clocks doing useful work.
- The relative loss contains no n at all: it depends only on the ratio of memory to message length. That one expression gives the whole table: $2/5 = 40\%$, $2/22 = 9.1\%$, $2/102 = 2.0\%$, $2/1002 = 0.2\%$.
- Rule of thumb: about $100m$ message bits for a 1% penalty. A $K = 9$ code has $m = 8$, so a 40-bit control message loses $8/48 = 17\%$ — which is exactly why control channels use tail-biting.
- **Why a ratio, not a quantity:** the wasted work is always the same size, exactly m stages, whether the message is 3 bits or a million. Flush overhead is a fixed tax, and a fixed tax only hurts small incomes.

The Decoding Problem

- The encoder walked one path through the trellis. Noise means the receiver cannot see which one.
- Maximum-likelihood decoding picks the codeword closest to what was received. Over a binary symmetric channel "closest" means smallest Hamming distance, because fewer bit flips is always the more probable explanation.
- A naive search would score all 2^L possible messages, which is hopeless for realistic L. Viterbi finds the same answer without ever listing them.

The insight Viterbi added

Paths that merge into the same state at the same time can be compared *immediately*, because from that state onwards they face exactly the same future. Only the better one can ever win, so the other is discarded on the spot.

Why "Nearest" Means "Most Likely"

On a BSC each bit flips independently with probability $p < \frac{1}{2}$. If $\mathbf{c}$ was sent and $\mathbf{r}$ arrived, exactly d_H bits flipped and $n - d_H$ did not, so the per-bit probabilities multiply:

$$P(\mathbf{r} \mid \mathbf{c}) = p^{\,d_H}(1-p)^{\,n-d_H} = \underbrace{(1-p)^n}_{\text{same for every } \mathbf{c}} \cdot \left(\frac{p}{1-p}\right)^{d_H}$$

- The first factor is a constant, so it cannot decide anything.
- $p < \frac{1}{2} \Rightarrow \frac{p}{1-p} < 1$, and a number below one shrinks as its exponent grows.
- Hence $\arg\max_{\mathbf{c}} P(\mathbf{r} \mid \mathbf{c}) = \arg\min_{\mathbf{c}} d_H(\mathbf{r}, \mathbf{c})$.

A theorem, not a heuristic

Counting bit flips *is* computing probability here. Each unit of distance costs one factor of $\frac{1-p}{p}$: our received word sits at $d_H = 1$ from the truth and $d_H = 4$ from its nearest rivals, so at $p = 0.01$ the right answer is $99^3 = 970{,}299$ times more likely. At $p = 0.1$ that margin collapses to $9^3 = 729$.

Why the Decoder Never Needs to Know p

Take logarithms — which is exactly what turns that product into the running *sum* that add–compare–select accumulates:

$$-\ln P(\mathbf{r} \mid \mathbf{c}) = \underbrace{-\,n \ln(1-p)}_{\text{constant}} + d_H(\mathbf{r}, \mathbf{c}) \cdot \underbrace{\ln \tfrac{1-p}{p}}_{>\,0}$$

- **Metrics add because logarithms turn products into sums.** A path metric *is* an accumulated log-likelihood, rescaled — that is where the "add" in add–compare–select comes from.
- **The value of p cancels out of the ranking.** It fixes the slope and the offset, and neither changes *which* path minimises. One hard-decision Viterbi decoder is optimal at every SNR, with no channel estimate at all.

The two edge cases

At $p = \frac{1}{2}$ the ratio is 1: every codeword is equally likely and the channel carries nothing at all. For $p > \frac{1}{2}$ the inequality reverses and ML would pick the *farthest* codeword — not a paradox, since inverting every received bit restores an ordinary channel with crossover $1 - p < \frac{1}{2}$.

Viterbi in Three Rules

Notation and start-up

$M(s)$, the *path metric* of state s, is the fewest bit disagreements any path reaching s has accumulated so far. The encoder starts with empty memory, so $M(00) = 0$ and $M(s) = \infty$ elsewhere — that is why only state 00 is alive at the first stage. The decoder stores one metric and one survivor per state, so four numbers and four pointers here, however long the message.

1. **Branch metric.** Count the positions where the received pair differs from what the branch would have sent: $d_H(r_t, \text{label})$.
2. **Add–compare–select.** Each state has two incoming branches. Add each predecessor's metric to its branch metric, keep the smaller total, and record which state that *survivor* came from.
3. **Traceback.** From the state where the path must end (00 for a terminated trellis), follow the recorded survivors backwards; their inputs give the decoded message.

$$M_t(s) = \min_{s' \to s} \Big[M_{t-1}(s') + d_H\big(r_t,\ \text{label}(s' \to s)\big) \Big]$$

Viterbi Example: What the Receiver Sees

Message	101
Transmitted	11 10 00 10 11
Channel	flips the 5th bit
Received	11 10 **1**0 10 11

- The received word 11 10 10 10 11 is *not* a valid codeword: no path through the trellis produces it.
- The decoder does not know which bit is wrong, or even how many are wrong.
- It only knows the trellis, so it looks for the path whose output differs from the received word in as few places as possible.
- Watch the final path metric: it will come out as 1, exactly the number of bit errors the decoder had to explain away.

Viterbi Stages 1–2

Stage 1, received $r_1 = 11$. Only state 00 is alive. Its two branches are labelled 00 and 11, giving branch metrics $d_H(11, 00) = 2$ and $d_H(11, 11) = 0$. So $M(00) = 2$ and $M(10) = 0$, and nothing has to be compared yet.

Stage 2, received $r_2 = 10$. States 00 and 10 are alive; still no state has two incoming branches.

From (metric)	Branch	Label	Metric $d_H(10, \cdot)$	Path metric at $t = 2$
00 (2)	$00 \xrightarrow{0} 00$	00	1	$2 + 1 = 3$
00 (2)	$00 \xrightarrow{1} 10$	11	1	$2 + 1 = 3$
10 (0)	$10 \xrightarrow{0} 01$	10	0	$0 + 0 = 0$
10 (0)	$10 \xrightarrow{1} 11$	01	2	$0 + 2 = 2$

State of play at $t = 2$

$M(00) = 3$, $M(01) = 0$, $M(10) = 3$, $M(11) = 2$. All four states are now occupied, so from here on every state gets two candidates and the real work begins.

Viterbi Stage 3: Add–Compare–Select

Received $r_3 = 10$ — this is the corrupted pair; the encoder actually sent 00.

Into state	Candidate A	Candidate B	Survivor	Came from
00	from 00: $3 + d_H(10, 00) = 3 + 1 = 4$	from 01: $0 + d_H(10, 11) = 0 + 1 = \mathbf{1}$	1	01
01	from 10: $3 + d_H(10, 10) = 3 + 0 = \mathbf{3}$	from 11: $2 + d_H(10, 01) = 2 + 2 = 4$	3	10
10	from 00: $3 + d_H(10, 11) = 3 + 1 = 4$	from 01: $0 + d_H(10, 00) = 0 + 1 = \mathbf{1}$	1	01
11	from 10: $3 + d_H(10, 01) = 3 + 2 = 5$	from 11: $2 + d_H(10, 10) = 2 + 0 = \mathbf{2}$	2	11

- Four comparisons, four survivors kept, four losing paths deleted permanently.
- Note that no path metric is zero any more: the corrupted pair forced every candidate to accumulate at least one mismatch. That is the decoder detecting damage without being told about it.

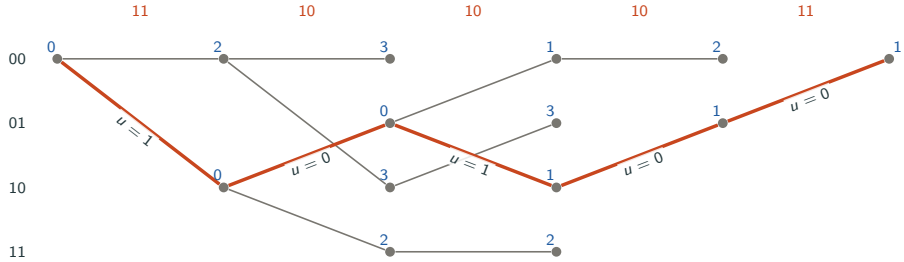
Viterbi Stages 4–5: Closing the Trellis

The last two inputs are known flush zeros, so only $u = 0$ branches exist and the trellis narrows back to state 00.

Stage	Into state	Candidate A	Candidate B	Survivor
4, $r_4 = 10$	00	from 00: $1 + d_H(10, 00) = \mathbf{2}$	from 01: $3 + d_H(10, 11) = 4$	2, from 00
4, $r_4 = 10$	01	from 10: $1 + d_H(10, 10) = \mathbf{1}$	from 11: $2 + d_H(10, 01) = 4$	1, from 10
5, $r_5 = 11$	00	from 00: $2 + d_H(11, 00) = 4$	from 01: $1 + d_H(11, 11) = \mathbf{1}$	1, from 01

Surviving path metric	$t = 0$	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
state 00	0	2	3	1	2	**1**
state 01	–	–	0	3	1	–
state 10	–	0	3	1	–	–
state 11	–	–	2	2	–	–

Traceback: Reading Out the Message



Orange pairs across the top are the received bits. Grey lines are the surviving branch into each state, the orange line is the traceback from terminal state 00, and the blue numbers are path metrics.

Decoded output

Traceback gives inputs $1, 0, 1, 0, 0$; dropping the flush bits leaves the message 101. The final metric 1 says the decoder explained the data with exactly one channel bit error, and repaired it with no retransmission.

Why Viterbi Is Affordable

Brute-force search

$$2^L \text{ candidate messages}$$

For $L = 100$ that is

$$2^{100} \approx 1.3 \times 10^{30}.$$

Impossible, at any clock speed.

Viterbi

$$(L + m) \times 2^m \times 2 \text{ operations}$$

For $L = 100$, $m = 2$:

$$102 \times 4 \times 2 = 816.$$

Linear in message length.

- Cost grows linearly with L but exponentially with memory m, so practical codes stop around $K = 7$ (64 states) or $K = 9$ (256 states).
- That single trade-off is why convolutional codes stayed short and shallow, and why the 1990s needed a different idea to get closer to capacity.

Reading the Complexity Count

That second figure counts the *branches* in the trellis, not the paths through it:

$$\underbrace{(L+m)}_{\text{stages: one per clock}} \times \underbrace{2^m}_{\text{states per stage}} \times \underbrace{2^k = 2}_{\text{branches per state}}$$

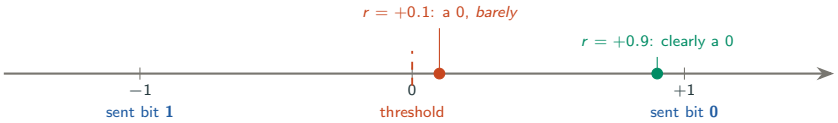
- $L+m$: one stage per message bit plus one per flush bit — the same clock count that set the termination penalty.
- 2^m: the width of a trellis column, and it is the *same* width at every stage, however long the message.
- $2^k = 2$: one input bit, so two branches leave each state and two merge into it — one add–compare–select each.
- n never appears — it scales the price of a branch, not the number of them. In general $(L/k+m) \times 2^\nu \times 2^k$.

Edges, not paths

Brute force counts paths, and there are 2^L; Viterbi counts edges, and there are 2^{m+1} per stage. Each stage doubles the surviving histories, then merges them back down to 2^m — the state summarises all of the past that the future can use. The two cancel exactly: the frontier never widens, so L multiplies instead of exponentiating.

What the Demodulator Actually Hands Over

BPSK sends one voltage per coded bit. **Mind the convention:** bit $0 \to +1\,\mathrm{V}$ and bit $1 \to -1\,\mathrm{V}$. The channel adds noise, so the receiver measures $r_i = s_i + n_i$ — a *real number*, not a bit.



- **Hard decision** reports only *which side* of the threshold the sample fell on. Both dots above become the single bit 0, and the decoder can no longer tell them apart.
- **Soft decision** passes the number through. $+0.9$ is strong evidence; $+0.1$ is almost a coin toss. That difference is exactly what lets one bit overrule another.

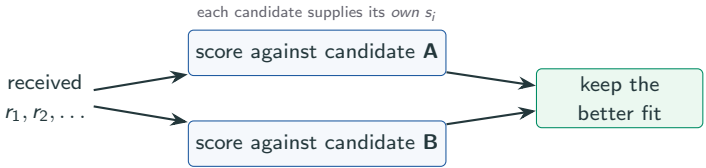
Why the order matters

Quantising happens *before* the decoder runs. Once the magnitude is discarded it can never be recovered, no matter how clever the decoder is. Soft decision costs 3–6 bits per sample instead of 1, and buys about 2 dB — roughly 37% less transmit power for the same BER.

Where the Score Comes From

The decoder never knows what was sent

That is the whole problem. What it *does* know is the **code** — every legal codeword, stored as the trellis. So it never asks "what was sent?", but of each candidate in turn: **"if this had been sent, how well would it explain what I received?"**



- The s_i in any metric come from **the candidate being scored**, never from the channel: no "truth" row, only hypotheses on the same evidence.
- **Hard:** the candidate supplies *bits*; count the mismatches. $\Rightarrow$ Hamming distance.
- **Soft:** the candidate supplies ± 1 *symbols*; correlate them with the samples. $\Rightarrow \sum_i r_i s_i$.
- In Viterbi the candidates are *branches*, and a branch label is exactly that branch's s_i — the same question, asked one stage at a time.

Where the Soft Metric Comes From

BPSK in AWGN: $r_i = s_i + n_i$, $s_i = \pm 1$, noise variance σ^2. Run the same maximum-likelihood argument on *this* channel.

Step 1 — how likely is a candidate? Gaussian noise turns ML into minimum **Euclidean** distance — the exact analogue of minimum Hamming distance on the BSC.

$$P(\mathbf{r} \mid \mathbf{s}) \;\propto\; \exp\Big(-\tfrac{1}{2\sigma^2}\textstyle\sum_i (r_i - s_i)^2\Big) \qquad \Longrightarrow \qquad \text{maximise } P \;\equiv\; \text{minimise } \textstyle\sum_i (r_i - s_i)^2$$

Step 2 — drop what cannot decide anything.

$$\sum_i (r_i - s_i)^2 = \underbrace{\sum_i r_i^2}_{\text{same for every candidate}} - 2\sum_i r_i s_i + \underbrace{\sum_i s_i^2}_{= n,\ \text{since } s_i = \pm 1}$$

Both outer terms are the same for every candidate, so neither can separate them: minimising distance $\equiv$ **maximising the correlation** $\sum_i r_i s_i$.

The soft branch metric

Add $+r_i$ where the branch would send $+1$, $-r_i$ where it would send -1; keep the *largest* total. Add–compare–select is unchanged — only the number being added differs.

Hard Fails, Soft Succeeds: Setting It Up

Two *legal codewords* compete — both genuine paths through our trellis, exactly $d_{\text{free}} = 5$ apart: $\mathbf{A} = 11\,10\,00\,10\,11$ (message 101) and $\mathbf{B} = 00\,00\,11\,10\,11$ (message 001). They *agree* at positions $4, 7, 8, 9, 10$, so only the five positions where they differ can decide anything.

Position	1	2	3	5	6
candidate **A** bits	1	1	1	0	0
candidate **B** bits	0	0	0	1	1
A would send s_i	-1	-1	-1	$+1$	$+1$
B would send s'_i	$+1$	$+1$	$+1$	-1	-1
receiver samples r_i	$+0.1$	$+0.1$	$+0.1$	$+0.9$	$+0.9$

- Rows 3–4 are rows 1–2 pushed through BPSK ($0 \to +1$, $1 \to -1$). **Each candidate supplies its own s_i**; neither row is "the truth", and the decoder cannot know which is.
- *If* **A** were sent, positions 1–3 went out as -1 and arrived at $+0.1$ — across the threshold, but barely; positions 5–6 went out as $+1$ and arrived at $+0.9$, decisively. And **B** is **A**'s complement here, so $s'_i = -s_i$ and the two soft scores will be exact opposites.

Hard Fails, Soft Succeeds: The Two Verdicts

Hard decision quantises first: all five samples are positive, so the magnitudes vanish and the decoder sees 0 0 0 0 0. It can only count mismatches — and smallest distance wins, so it returns **B**, message 001.

Position	1	2	3	5	6	
hard decision sees	0	0	0	0	0	
vs **A** 1, 1, 1, 0, 0	differs	differs	differs	same	same	$d_H = \mathbf{3}$
vs **B** 0, 0, 0, 1, 1	same	same	same	differs	differs	$d_H = \mathbf{2}$

Soft decision never quantises. It correlates the samples with each candidate's own symbols; largest correlation wins, so it returns **A**, message 101.

$$\text{score}(\mathbf{A}) = \underbrace{-0.1 - 0.1 - 0.1}_{\text{leans away from } \mathbf{A}} \underbrace{+0.9 + 0.9}_{\text{leans towards } \mathbf{A}} = +\mathbf{1.5}, \qquad \text{score}(\mathbf{B}) = -\text{score}(\mathbf{A}) = -\mathbf{1.5}$$

A is the codeword that was actually sent

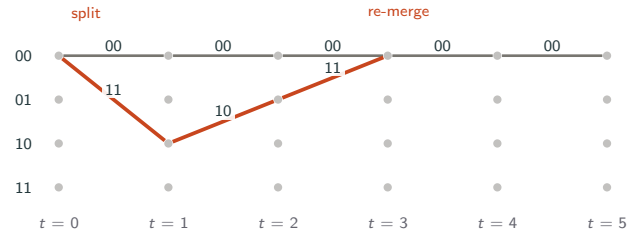
Hard counted **three votes against two** and got it **wrong**; soft weighed **0.3** of evidence **against 1.8** and got it **right** — because the three outvoting samples were exactly the ones nearest the threshold.

Free Distance: The Question It Has to Answer

A block code has 2^k codewords of equal length: compare every pair, read off $d_{\min}$. A convolutional code's list of codewords never ends, so $d_{\min}$ needs replacing by something computable.

Two facts that collapse an infinite comparison into one finite search

(a) Viterbi fails in events, erring only when its survivor **splits** from the true path and later **re-merges**; outside that stretch both paths agree bit for bit. **(b) Linearity:** $d_H(\mathbf{c}_1, \mathbf{c}_2) = \text{weight}(\mathbf{c}_1 \oplus \mathbf{c}_2)$, and that difference is *itself* a legal path leaving 00 and returning. So define d_{free} = **the smallest output weight of any path that leaves** 00 **and returns.**



The grey all-zero path is the reference. The red excursion splits at $t = 0$, re-merges at $t = 3$, and emits 11 10 11 — **five** 1s.

Those five bits are the whole contest: noise must corrupt enough of them to make red the better explanation. Every pair of paths in the trellis differs by a shifted copy of some picture like this.

Free Distance: Where the Number 5 Comes From

Excursions can be arbitrarily long, so the search still looks endless. Three readings off the branch table close it in one step.

No excursion can weigh less than 5

- **Leaving** 00 **costs 2.** The only branch out is 1/**11**, landing in state 10.
- **Re-entering** 00 **costs 2.** The only branch in, apart from the self-loop, is 0/**11** from state 01.
- **The middle costs at least 1.** Whatever happens in between, the path must still travel from 10 to 01, and *both* branches leaving 10 emit a single 1 (0/**10** and 1/**01**).

Every excursion therefore weighs $\geq 2 + 1 + 2 = \mathbf{5}$, and the input 1 0 0 hits that bound exactly — so the bound is tight and the search is over. Nothing infinite was ever enumerated.

Every excursion up to five branches

Input	Emits	Weight
100	11 10 11	**5**
1100	11 01 01 11	6
10100	11 10 00 10 11	6
11100	11 01 10 01 11	7

Longer detours only add weight: the one weight-0 branch off the all-zero path is 01 $\xrightarrow{1/00}$ 10, and getting back to 01 costs 1. No loop is free.

$d_{\text{free}} = 5$, reached by exactly *one* excursion, so the multiplicity is $N_{\text{free}} = 1$ (then 2 events at weight 6, 4 at weight 7, 8 at weight 8, ...). Its output 11 10 11 is just $g_1 = 111$ and $g_2 = 101$ interleaved — the encoder's impulse response, which is what an isolated 1 always produces.

What d_{free} Buys: Capability and Error Rate

Why $t = \lfloor (d_{\text{free}} - 1)/2 \rfloor$

Inside one error event the true path and its lightest rival disagree in exactly $d_{\text{free}} = 5$ coded bits. Noise decides which of the two the received word ends up nearer to:

- Flip **2** of those five: 2 disagreements with the truth, 3 with the rival — the truth still wins.
- Flip **3**: now 3 against 2, and the decoder confidently returns the wrong message.

It is the block-code sphere argument applied *per event*: $t = \lfloor (5-1)/2 \rfloor = 2$. *Two Errors Repaired, Three Errors Not* plays out both cases on our own codeword.

Why it sets the slope of the error curve

To fail, the channel must land $t + 1 = 3$ flips inside the five positions of one minimum-weight event. On a BSC with crossover p:

$$P_{\text{event}} \approx N_{\text{free}} \binom{d_{\text{free}}}{t+1} p^{\,t+1}$$

$$= \binom{5}{3} p^3 = 10\, p^3.$$

At $p = 10^{-2}$ that is 10^{-5} — three decades below the raw channel, bought with one redundant bit per message bit.

The one thing to carry away

d_{free} does not shift the error curve, it sets its **exponent** $t + 1$. Two more units of d_{free} buy one more power of p — at $p = 10^{-2}$, another two decades of BER at the same transmit power. That is the whole reason generators are found by computer search: K and R fix the cost, and d_{free} is the only number left to maximise.

Free Distance: Reading the Consequences

Standard rate-$1/2$ codes

K	States	Generators	d_{free}	t	Gain
3	4	$(7,5)_8$	5	2	4.0 dB
5	16	$(23,35)_8$	7	3	5.4 dB
7	64	$(133,171)_8$	10	4	7.0 dB
9	256	$(561,753)_8$	12	5	7.8 dB

Gain $= 10\log_{10}(R\,d_{\text{free}})$, the asymptotic soft-decision coding gain. At a working BER of 10^{-5} the realised figure is 1–2 dB lower, and hard decisions give up about 2 dB more.

What t actually promises

t errors *per error event*, not per message. Errors 500 bits apart are separate excursions, each with its own budget — there is no global allowance the way a block code has one per block.

Where the generators come from

Nobody derives $(133,171)_8$ by hand. They are the winners of an exhaustive search over all 64-state encoders, ranked by d_{free} first and multiplicity N_{free} second.

Why this table matters more than its numbers

Each row quadruples the state count — and the decoder's work per bit — for 1–1.5 dB. d_{free} climbs only **linearly** in K while complexity climbs as 2^{K-1}, and in dB the return is logarithmic on top of that. $K = 3 \to 9$ is 64× the hardware for 3.8 dB — which is why the road to the Shannon limit had to leave convolutional codes behind.

Two Errors Repaired, Three Errors Not

Terminating after our 3-bit message leaves eight codewords, of weights $0, 5, 5, 5, 6, 6, 6, 7$. The lightest is $5 = d_{\text{free}}$, so $d_{\min} = 5$ and $t = 2$. Our $\mathbf{c} = 11\,10\,00\,10\,11$ and its nearest rival $\mathbf{c}' = 00\,00\,11\,10\,11$ differ in positions $1, 2, 3, 5, 6$ — exactly one minimum-weight error event.

Channel	Received	d_H to c	d_H to c′	Viterbi returns	Metric
flips bits 5, 6	11 10 11 10 11	**2**	≥ 3	$\mathbf{u} = 101$ **right**	2
flips bits 1, 2, 3	00 00 00 10 11	3	**2**	$\mathbf{u}' = 001$ **wrong**	2

Read those two metrics again

Both runs finish with a final metric of 2; one decoded correctly and the other did not. The decoder is not broken and the metric is not lying — with three errors the wrong path genuinely *is* the better explanation of what arrived. Nothing inside the code can tell the two outcomes apart.

Why Every Convolutional Code Ships with a CRC

The asymmetry

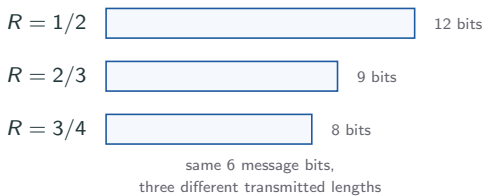
A block code has a reject test: syndrome $\neq$ **0** means "not a codeword". A convolutional code has none. **Every** received sequence has a nearest path, so Viterbi always returns a well-formed message and can never say "I don't know".

- The CRC is computed over the information bits *before* encoding, so it travels as protected payload. The receiver corrects first and checks second.
- When Viterbi does fail it follows a wrong excursion spanning several stages, so its output errors arrive **in bursts**. A parity bit would miss any even-length burst — a CRC is required, not merely preferred.
- Hence every LTE and 5G transport block carries both a strong FEC code and a CRC — only the CRC can produce a verdict.

Puncturing: One Encoder, Many Rates

Your link is not one channel. Next to the access point it is clean; three rooms away it is not. Those two situations want *different amounts of protection*.

- **Strong signal:** parity is mostly wasted throughput. You want $R = 3/4$.
- **Weak signal:** you need all the protection you can get. You want $R = 1/2$.
- So a real link needs a *family* of rates, switchable in milliseconds as you move.



The trick

Do *not* build a code per rate — each would need its own trellis, and therefore its own Viterbi engine. Build **one** rate-1/2 *mother code*, and raise the rate by simply **not transmitting** some of its output bits, on a schedule both ends know.

Puncturing: Why Skipping Bits Still Works

Spelling a name over a bad phone line

Rate 1/2 is "**S** as in Sierra, **M** as in Mike, **I** as in India" — every letter gets a confirming word. On a decent line you speed up: "**S** as in Sierra, **M**, **I** as in India". Still unambiguous, because the listener's sense of a plausible name fills the gaps. **That sense is the trellis.**

- You would expect deleting bits to break the decoder. It does not: a branch metric is a *sum of per-bit comparisons*, so a bit that was never sent contributes 0 to every branch equally.
- In soft decision this is exact. The LLR of a punctured position is 0 — "perfectly 50/50, no information" — which is precisely the receiver's true situation.
- **So the trellis never changes.** Same states, same branches, same labels, same algorithm; only the *amount of evidence* per stage changes.

Puncturing: The Pattern, Concretely

Pattern $P = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

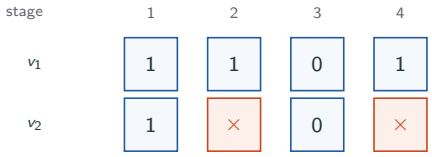
Row 1 is v_1, row 2 is v_2, columns are consecutive stages, and the pattern repeats. A 1 means transmit, a 0 means delete.

- Column 1: send both.
- Column 2: send v_1, delete v_2.

Over one period: 2 bits in, 4 out of the encoder, 1 deleted, 3 transmitted.

$$R = \tfrac{2}{3}$$

Punching the holes in our own stream — the name is literal.



Rate 1/2 would have sent 11 10 00 10 — eight bits. After punching out the two shaded positions the transmitter sends

$$11 \cdot 1 \cdot 00 \cdot 1$$

4 input bits → 6 transmitted bits instead of 8.

Why it matters: this trades protection for throughput *without changing hardware*, which is how Wi-Fi offers 1/2, 2/3 and 3/4 from a single $K = 7$ encoder and one Viterbi engine.

Puncturing: What the Decoder Actually Does

P has n rows and p columns and then repeats. Each period consumes kp information bits and transmits one bit per 1 in P:

$$R = \frac{k\,p}{w(P)}, \qquad w(P) = \text{number of ones in } P$$

- Ours: $k = 1$, $p = 2$, $w = 3 \Rightarrow R = 2/3$. Rate 3/4 needs $p = 3$, $w = 4$. One mother code reaches *any* rate p/w with $w \leq np$.
- A punctured bit is not a missing bit — it is a bit received with **zero reliability**, at a position both ends already know. The decoder inserts a placeholder carrying no evidence: LLR $= 0$ soft, or a branch-metric contribution of 0 hard.

Why one engine serves every rate

Same states, same branches, same survivors, same traceback — only the branch metric at punctured stages has fewer terms. Depuncturing happens in the buffer *before* the decoder, so changing rate is a table lookup, not a hardware change.

What Puncturing Costs, on Our Own Code

Deleting coded bits can only shrink an error event, so d_{free} can only fall. How far depends on *where* the event starts relative to the period — and d_{free} is the minimum over all phases. Our lightest event is input $1, 0, 0$, emitting 11 10 11 with weight 5:

Event starts on	Stage 1	Stage 2	Stage 3	Surviving weight
a "keep both" column	$11 \to 11$ (2)	$10 \to 1$ (1)	$11 \to 11$ (2)	**5**
a "v_1 only" column	$11 \to 1$ (1)	$10 \to 10$ (1)	$11 \to 1$ (1)	**3**

So $d_{\text{free}} = 3$ and $t = 1$: going from $R = 1/2$ to $R = 2/3$ **halves** the correction capability. For the deployed $K = 7$ code, d_{free} falls $10 \to 6 \to 5$ as the rate rises $1/2 \to 2/3 \to 3/4$, so t falls $4 \to 2 \to 2$.

Two consequences

Patterns are tabulated, not invented: since the worst phase sets d_{free}, the standard patterns come from exhaustive searches that maximise it. **Nest the patterns and you get HARQ:** if the bits kept at $3/4$ are a subset of those kept at $2/3$, the transmitter can send the small set first and, on failure, send only the extra bits — lowering the rate with no re-encoding.

Convolutional Codes in Real Systems

System	Code	Why that choice
Voyager (deep space)	$K = 7$, $R = 1/2$, plus Reed–Solomon	No retransmission at light-hour distances
GSM speech	$K = 5$, punctured, interleaved	Spreads fading bursts across frames
802.11a/g/n	$K = 7$, $(133, 171)_8$, punctured	One decoder serves every data rate
LTE control channel	Tail-biting $K = 7$	Tiny messages; no flush overhead

Pattern worth noticing

Convolutional codes dominate where messages stream continuously and decoders must be cheap. They lose ground where blocks are long and every fraction of a dB matters.

Practice

Use the same $(7, 5)_8$, $K = 3$ encoder throughout.

1. Encode the message $\mathbf{u} = 110$, including the two flush bits. Give the state sequence and the transmitted bits.
2. The receiver gets 11 10 00 10 11 with no errors. What is the final path metric, and why?
3. A 3-bit message cost 10 transmitted bits. What is the effective rate, and what would it be for a 500-bit message?

Practice: Solutions

1. Encoding $\mathbf{u} = 110$:

Step	u_i	**State**	v_1	v_2	**Next state**
1	1	00	1	1	10
2	1	10	0	1	11
3	0	11	0	1	01
4	0 (flush)	01	1	1	00
5	0 (flush)	00	0	0	00

State sequence $00 \to 10 \to 11 \to 01 \to 00 \to 00$, transmitted 11 01 01 11 00.

2. The received word is a valid codeword, so the correct path accumulates a branch metric of 0 at every stage and the final path metric is 0. A zero metric means the decoder found an explanation with no bit errors at all.

3. $R_{\text{eff}} = 3/10 = 0.30$ for the 3-bit message, and $500/(2 \times 502) = 0.498$ for the 500-bit message. The flush overhead is fixed, so it becomes negligible as messages grow.

How to Choose the Coding Idea

Situation	Natural coding choice	Reasoning
Wired frame with cheap retransmission	CRC + ARQ	Detect reliably, resend bad frames
One-bit/simple teaching example	Parity or repetition	Shows redundancy with minimal machinery
Small block correction example	Hamming code	Corrects one error efficiently compared with repetition
Streamed wireless/satellite link	Convolutional code with Viterbi	Feedback may be delayed or costly
Same encoder, several link rates	Punctured convolutional code	One decoder serves every rate the standard offers

Common Confusions

- **Detection is not correction.** A CRC can be excellent at rejecting corrupted frames but still does not tell the receiver exactly what to fix.
- **More redundancy is not automatically better.** It improves reliability only relative to the channel and decoder, and it reduces net data rate.
- **A code rate is not a bit rate.** Rate $R = k/n$ is a ratio; actual throughput also depends on symbol rate, modulation, retransmissions, and overhead.

Common Confusions: Convolutional Codes

- **Constraint length is not block length.** $K = 3$ does not mean the message is 3 bits; it is how far back the encoder remembers. Messages can be thousands of bits long.
- d_{free} **is not a budget for the whole message.** Correcting two errors means two per error event. Errors that are far apart in the trellis are handled independently.
- **Viterbi does not try every path.** It keeps one survivor per state, which is exactly why its cost is linear in message length instead of exponential.
- **Puncturing does not change the encoder.** The same shift register and the same trellis are used; the deleted positions simply enter the decoder as "no information".

Summary I: Block Codes

- Channel coding adds structured redundancy to fight noise.
- Error detection asks whether something went wrong; error correction tries to infer the original codeword.
- Code rate $R = k/n$ measures redundancy cost.
- Hamming distance and $d_{\min}$ explain detection and correction capability.
- Hamming $(7, 4)$ corrects one bit error because $d_{\min} = 3$.
- CRC is a strong detection method based on polynomial division.

Summary II: Convolutional Codes

- A convolutional encoder has memory, so it is a state machine, and its possible histories form a trellis. Our $(7,5)_8$ example turned 101 into 11 10 00 10 11.
- In general k bits enter and n leave each step: $R = k/n$, $K = m + 1$, 2^{ν} states. We used $k = 1$, $n = 2$, $m = 2$.
- Viterbi decoding keeps one survivor per state and finds the most likely path in time linear in message length. It repaired a one-bit error in that stream and reported a final metric of 1.
- d_{free}, puncturing, and tail-biting are the knobs that adapt a single convolutional engine to many links and rates.
- Soft-decision decoding buys about 2 dB for the cost of a few quantisation bits, because the decoder keeps the confidence the demodulator measured.
- Convolutional codes dominate where messages stream continuously and decoders must stay cheap: Voyager, GSM, 802.11a/g/n, and LTE control channels all use the same machinery.

Appendix: First-Exposure Terminology I

Term	Meaning for this module
Channel coding	Adding structured redundancy so a receiver can detect or correct channel errors.
FEC	Forward error correction: correction by the receiver without waiting for retransmission.
ARQ	Automatic repeat request: detection plus retransmission.
Dataword	Original k-bit block before channel encoding.
Codeword	n-bit encoded block transmitted over the channel.
Code rate	Ratio $R = k/n$, measuring useful bits per transmitted coded bit.
Syndrome	A parity-check result used to decide whether and where an error likely occurred.

Appendix: First-Exposure Terminology II

Term	Meaning for this module
Hamming distance	Number of positions in which two equal-length binary words differ.
Minimum distance	Smallest Hamming distance between any two valid codewords.
Parity bit	Redundant bit chosen to make the total number of ones even or odd.
Cyclic code	Code with algebraic structure where cyclic shifts of codewords remain codewords.
CRC	Cyclic redundancy check; error detection by polynomial division and remainder checking.
Convolutional code	Stream code whose output depends on current and previous input bits.
Trellis	State-time diagram used to represent possible convolutional-code paths.
Viterbi decoding	Decoding method that selects the most likely path through a trellis.

Appendix: First-Exposure Terminology III

Term	Meaning for this module
(n, k, m) code	k bits in and n out per step, memory order m: rate $R = k/n$, $K = m + 1$, 2^{ν} states with $\nu = \sum_p m_p$. Here $k = 1$, $n = 2$, $m = \nu = 2$.
Constraint length K	$m + 1$, the number of input bits each output bit depends on.
Generator vector	Tap pattern of one output line, usually written in octal, e.g. $(7, 5)_8$.
Branch metric	Distance between the received symbols of one stage and what a branch would have sent.
Path metric	Running sum of branch metrics along a path through the trellis.
Survivor	The better of the two paths merging into a state; the other is discarded permanently.
Free distance d_{free}	Smallest output weight of any path that leaves state 00 and returns; the convolutional analogue of $d_{\min}$.
Puncturing	Deleting selected coded bits by a fixed pattern to raise the rate without changing the encoder.
Tail-biting	Ending the trellis in the same state it started, avoiding flush-bit overhead.
Soft decision	Decoding from real-valued samples or LLRs instead of hard $0/1$ decisions; worth about 2 dB.
LLR	Log-likelihood ratio; sign gives the decided bit, magnitude gives the confidence.

Appendix: References

All definitions, formulas, examples, and reproduced figures in this deck are drawn from the following sources; the Chapter 10 figures are reproduced from Forouzan.

- Local module: *Short Module: Source Coding, Channel Coding, and Joint Source–Channel Coding*, `Source and Channel Coding.pdf`, pp. 5–8.
- B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013, Ch. 10; existing course image extraction in `images/chapter10-error-codes/raw`.
- R. W. Hamming, "Error Detecting and Error Correcting Codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950. Local file: `references/hamming-1950-error-detecting-correcting.pdf`.
- D. J. Costello Jr. and G. D. Forney Jr., "Channel Coding: The Road to Channel Capacity," *Proceedings of the IEEE*, vol. 95, no. 6, pp. 1150–1177, 2007. Local file: `references/costello-forney-2007-road-to-channel-capacity.pdf`.

Questions?